



Integration Testing

Testing that the system functions cohesively

In the previous videos...



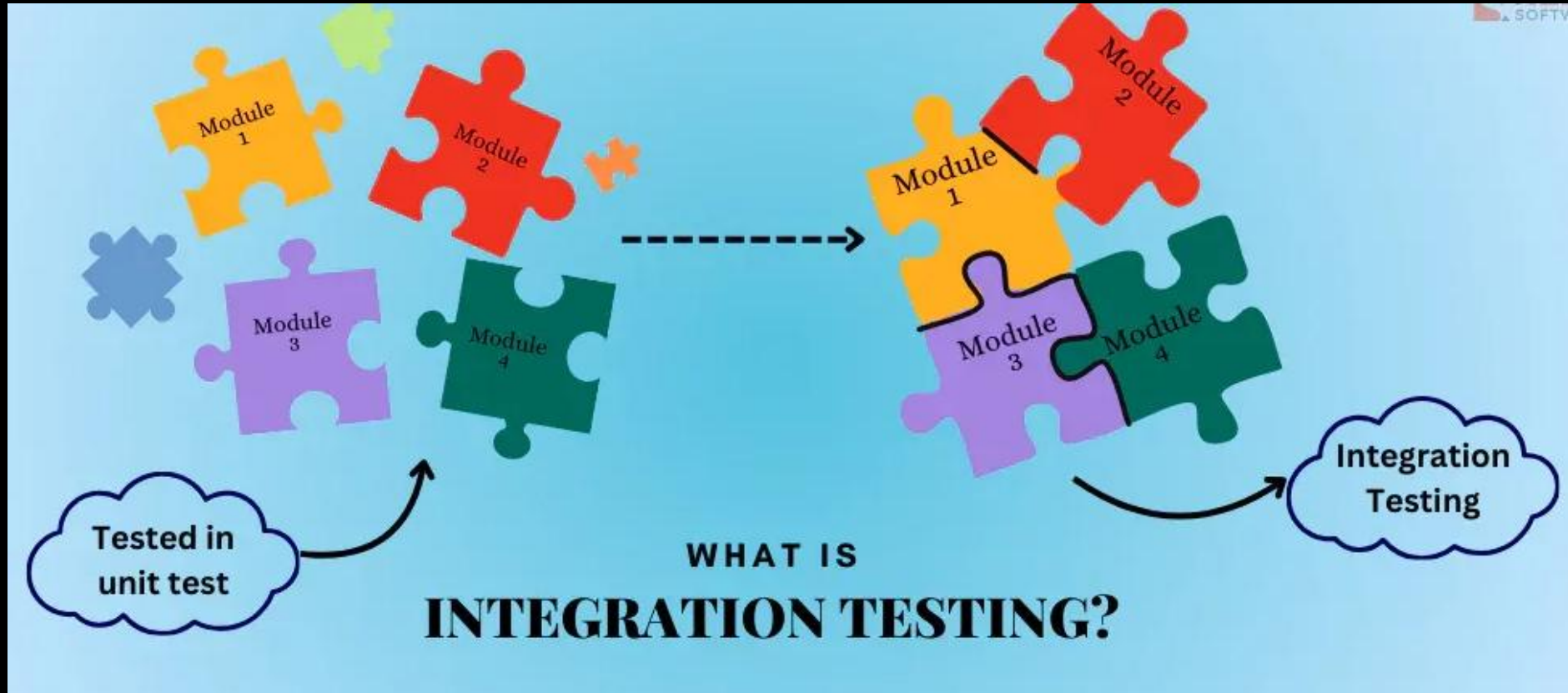
In this video...



What is Integration Testing?

- Software-testing approach where application components and modules are joined and tested
- This is done to evaluate how well they work together
- To ensure the parts of the program communicate with each other and interact appropriately

How does it differ from unit testing?



What do we classify as parts?

- The 2 major parts in this system:
 1. Backend (API):
 - Logic
 - Processing
 2. Frontend:
 - GUI
 - User interaction
- Ensure these 2 layers communicate and exchange data correctly

Framework



Testing for correct result

```
describe("Calculator API", () => {
  test("should return the correct result", async () => {

    // Send a request to the backend endpoint
    const response = await fetch("http://localhost:5000/example-endpoint", {
      method: "POST", //[GET, PUT, PATCH]
      headers: {
        "Content-Type": "application/json"
      },

      // Data being sent to the API
      body: JSON.stringify({
        key: "value"
      })
    });

    // Convert the response into JSON
    const data = await response.json();

    // Check if the request was successful
    expect(response.ok).toBe(true);

    // Check if the returned data is correct
    expect(data.exampleProperty).toBe("expected value");
  });
});
```


Testing for error result

```
describe("Calculator API", () => {
  test("should return an error for invalid input", async () => {

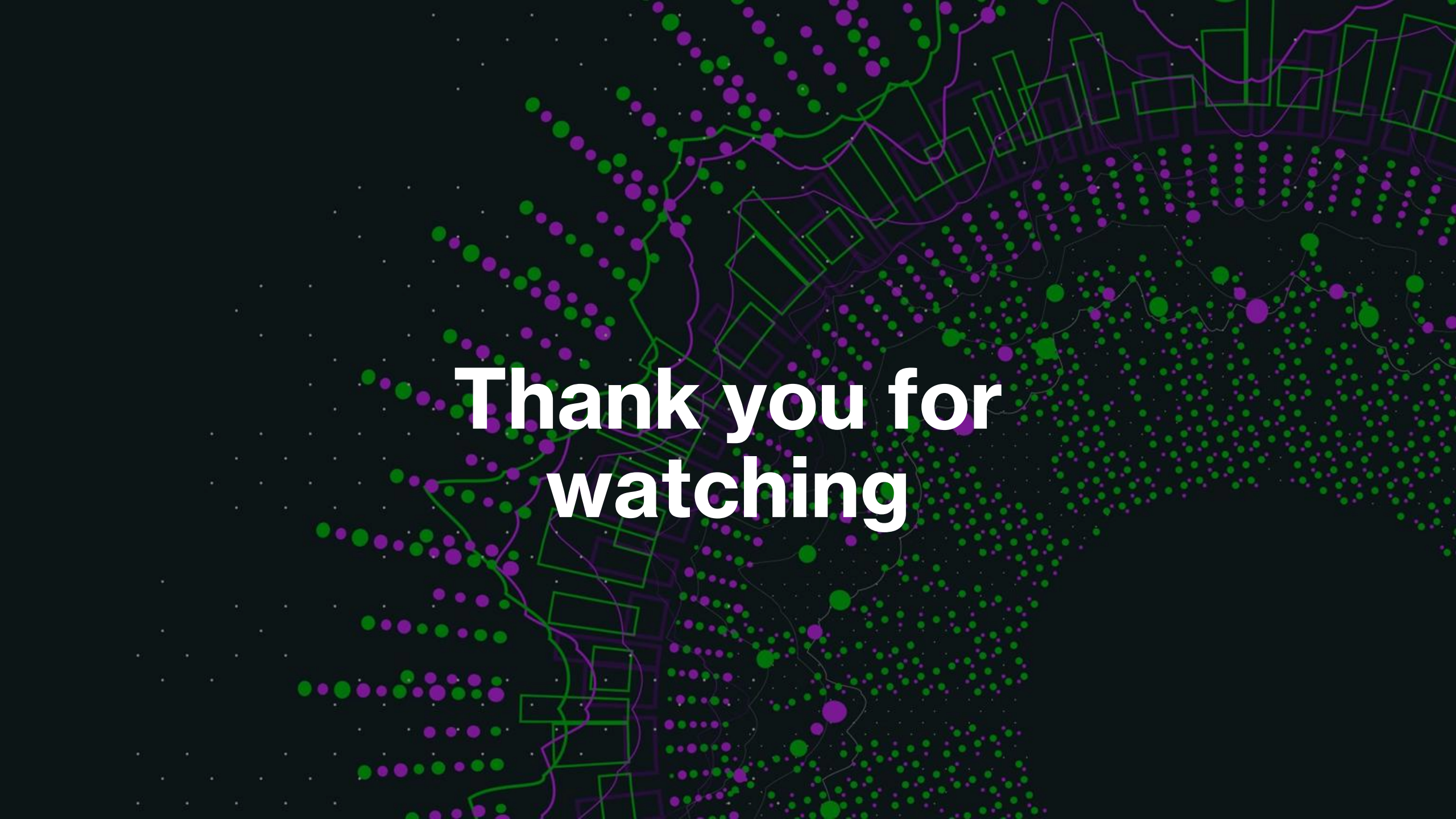
    // Send a request with invalid data
    const response = await fetch("http://localhost:5000/example-endpoint", {
      method: "POST", // [GET, PUT, PATCH]
      headers: {
        "Content-Type": "application/json"
      },

      body: JSON.stringify({
        key: "" // Empty field to cause an error
      })
    });

    // Convert the response into JSON
    const data = await response.json();

    // Check if the request failed
    expect(response.ok).toBe(false);

    // Check if the correct error message was returned
    expect(data.error).toBe("Invalid input");
  });
});
```

The background is a black field filled with a complex pattern of green and purple dots, lines, and rectangles. The dots are arranged in various clusters and trails. The lines are thin and wavy, connecting some of the dots. The rectangles are thin and appear to be floating or overlapping the other elements. The overall effect is a dense, abstract, and somewhat chaotic visual field.

**Thank you for
watching**